

① Find the tangent plane to the unit sphere $x^2 + y^2 + z^2 = 1$ at an arbitrary point (x_0, y_0, z_0) .

Unit sphere $\underbrace{x^2 + y^2 + z^2}_{= F(x, y, z)} = 1$ is the level surface $\{(x, y, z) \in \mathbb{R}^3 \mid F(x, y, z) = 1\}$

Gradient $\perp$ Surface level
Take (x, y, z) on the plane $\Rightarrow \nabla F(x_0, y_0, z_0) \cdot (x - x_0, y - y_0, z - z_0) = 0$

$$\nabla F(x, y, z) = \left(\frac{\partial F}{\partial x}, \frac{\partial F}{\partial y}, \frac{\partial F}{\partial z} \right) = (2x, 2y, 2z)$$

$$\Rightarrow \nabla F(x_0, y_0, z_0) = (2x_0, 2y_0, 2z_0)$$

$$\nabla F(x_0, y_0, z_0) \cdot (x - x_0, y - y_0, z - z_0) = 0$$

$$2x_0(x - x_0) + 2y_0(y - y_0) + 2z_0(z - z_0) = 0 \quad / : 2$$

$$x_0(x - x_0) + y_0(y - y_0) + z_0(z - z_0) = 0$$

$$x_0x - x_0^2 + y_0y - y_0^2 + z_0z - z_0^2 = 0$$

$$x_0x + y_0y + z_0z - \underbrace{(x_0^2 + y_0^2 + z_0^2)}_1 = 0$$

1

$$x_0x + y_0y + z_0z = 1$$

② Let $x, y, f: \mathbb{R}^2 \rightarrow \mathbb{R}$ and $f(x, y) = f(x(u, v), y(u, v))$. Prove that $\frac{\partial f}{\partial u} = \frac{\partial f}{\partial x} \frac{\partial x}{\partial u} + \frac{\partial f}{\partial y} \frac{\partial y}{\partial u}$ and $\frac{\partial f}{\partial v} = \frac{\partial f}{\partial x} \frac{\partial x}{\partial v} + \frac{\partial f}{\partial y} \frac{\partial y}{\partial v}$

Let $f: \mathbb{R}^2 \rightarrow \mathbb{R}, f(a, b)$

Let $g: \mathbb{R}^2 \rightarrow \mathbb{R}^2, g(u, v) = (a(u, v), b(u, v))$

Take $F(u, v) = f(g(u, v))$

$\nabla f = \left(\frac{\partial f}{\partial a}, \frac{\partial f}{\partial b} \right)$ - gradient of f

$J_g = \frac{\partial g}{\partial (u, v)} = \begin{pmatrix} \frac{\partial a}{\partial u} & \frac{\partial a}{\partial v} \\ \frac{\partial b}{\partial u} & \frac{\partial b}{\partial v} \end{pmatrix}$ - Jacobian of g

Chain Rule: $\nabla F = \nabla \varphi \cdot Jg$

$$\Rightarrow \begin{pmatrix} \frac{\partial F}{\partial u} & \frac{\partial F}{\partial v} \end{pmatrix} = \begin{pmatrix} \frac{\partial \varphi}{\partial a} & \frac{\partial \varphi}{\partial b} \end{pmatrix} \begin{pmatrix} \frac{\partial a}{\partial u} & \frac{\partial a}{\partial v} \\ \frac{\partial b}{\partial u} & \frac{\partial b}{\partial v} \end{pmatrix}$$

$$\begin{pmatrix} \frac{\partial F}{\partial u} & \frac{\partial F}{\partial v} \end{pmatrix} = \begin{pmatrix} \frac{\partial \varphi}{\partial a} \frac{\partial a}{\partial u} + \frac{\partial \varphi}{\partial b} \frac{\partial b}{\partial u} & \frac{\partial \varphi}{\partial a} \frac{\partial a}{\partial v} + \frac{\partial \varphi}{\partial b} \frac{\partial b}{\partial v} \end{pmatrix}$$

Rename $a \rightarrow x, b \rightarrow y$

$$\Rightarrow \begin{cases} \frac{\partial \varphi}{\partial u} = \frac{\partial \varphi}{\partial x} \frac{\partial x}{\partial u} + \frac{\partial \varphi}{\partial y} \frac{\partial y}{\partial u} \\ \frac{\partial \varphi}{\partial v} = \frac{\partial \varphi}{\partial x} \frac{\partial x}{\partial v} + \frac{\partial \varphi}{\partial y} \frac{\partial y}{\partial v} \end{cases}$$

③ Consider the quadratic function $\varphi: \mathbb{R}^2 \rightarrow \mathbb{R}, \varphi(x, y) = \frac{1}{2}(x^2 + by^2)$ with $b > 0$ and the following gradient descent method for finding its minimum: $(x_{k+1}, y_{k+1}) = (x_k, y_k) - \Delta_k \nabla \varphi(x_k, y_k)$, with step size $\Delta_k > 0$.

a) Find the step size Δ_k using linear search, such that it minimizes the function:

$$\varphi(\Delta_k) = \varphi(x_{k+1}, y_{k+1}) = \varphi((x_k, y_k) - \Delta_k \nabla \varphi(x_k, y_k)).$$

$$\varphi(x, y) = \frac{1}{2}(x^2 + by^2)$$

$$\nabla \varphi(x, y) = (x, by)$$

$$\begin{cases} x_{k+1} = x_k - \Delta_k x_k = (1 - \Delta_k) x_k \\ y_{k+1} = y_k - \Delta_k b y_k = (1 - \Delta_k b) y_k \end{cases}$$

$$\varphi(\Delta_k) = \frac{1}{2} [(1 - \Delta_k)^2 x_k^2 + b(1 - \Delta_k b)^2 y_k^2]$$

$$\parallel$$

$$\varphi(x_{k+1}, y_{k+1})$$

$$\frac{d\varphi}{d\Delta_k} = \frac{1}{2} [2(1 - \Delta_k)(-1) \cdot x_k^2 + b \cdot 2(1 - \Delta_k b)(-b) y_k^2] =$$

$$= -(1 - \Delta_k) x_k^2 - b^3 (1 - \Delta_k b) y_k^2$$

$$\frac{d\varphi}{d\Delta_k} = 0$$

$$\Rightarrow \cancel{(1 - \Delta_k)} x_k^2 + b^3 (1 - \Delta_k b) y_k^2 = 0$$

$$x_K^2 - \Delta_K x_K^2 + b^3 y_K^2 - \Delta_K b^4 y_K^2 = 0$$

$$x_K^2 + b^3 y_K^2 = \Delta_K (x_K^2 + b^4 y_K^2)$$

$$\Rightarrow \Delta_K = \frac{x_K^2 + b^3 y_K^2}{x_K^2 + b^4 y_K^2}$$

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 #given function
5 def f(x, y, b): 1 usage
6     return 0.5 * (x ** 2 + b * y ** 2)
7
8 #compute the gradient of the function
9 def grad_f(x, y, b): 1 usage
10     return np.array([x, b * y])
11
12 #compute optimal step using the formula from point a)
13 def get_optimal_step(x, y, b): 1 usage
14     num = x ** 2 + (b ** 3) * (y ** 2)
15     den = x ** 2 + (b ** 4) * (y ** 2)
16
17     # avoid division by zero if we are already at (0,0)
18     if den < 1e-15:
19         return 0
20     return num / den
21

```

```

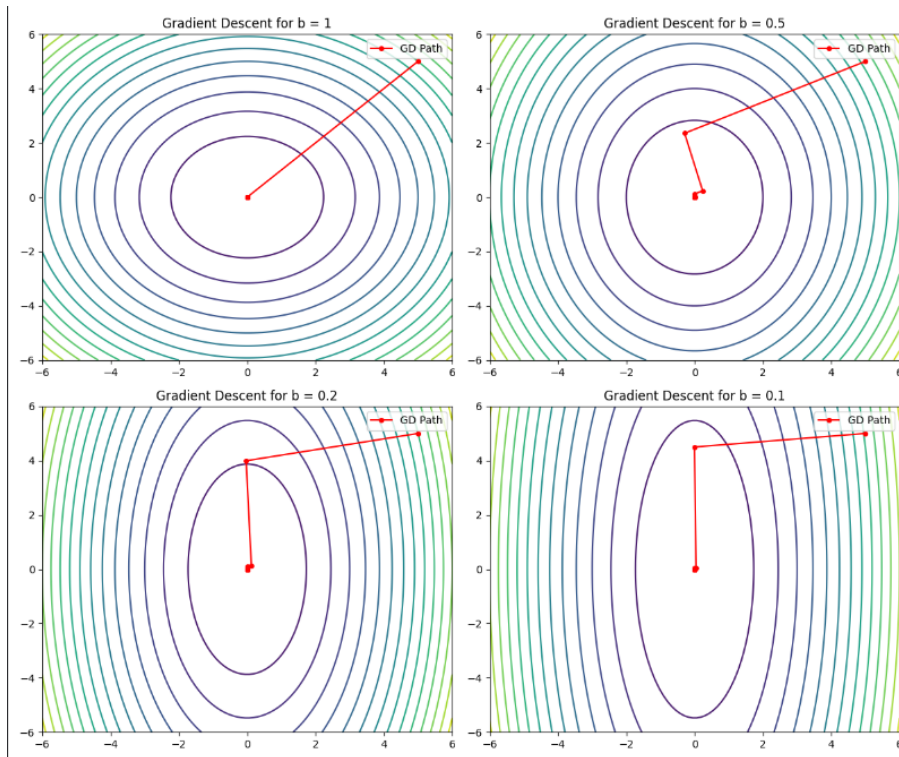
22 #main loop of the program
23 def gradient_descent(b, iterations=20, start_pos=(5.0, 5.0)): 1 usage
24     path = [start_pos]
25     curr_x, curr_y = start_pos
26
27     for _ in range(iterations):
28         s_k = get_optimal_step(curr_x, curr_y, b)
29         grad = grad_f(curr_x, curr_y, b)
30         curr_x, curr_y = np.array([curr_x, curr_y]) - s_k * grad
31         path.append((curr_x, curr_y))
32
33     return np.array(path)
34

```

```

35
36 #plotting
37 b_values = [1, 1 / 2, 1 / 5, 1 / 10]
38 fig, axes = plt.subplots( nrows: 2, ncols: 2, figsize=(12, 10))
39 axes = axes.flatten()
40
41 for i, b in enumerate(b_values):
42     path = gradient_descent(b)
43
44     #grid for contours
45     x = np.linspace(-6, stop: 6, num: 100)
46     y = np.linspace(-6, stop: 6, num: 100)
47     X, Y = np.meshgrid(x, y)
48     Z = f(X, Y, b)
49
50     axes[i].contour(X, Y, Z, levels=15, cmap='viridis')
51     axes[i].plot(path[:, 0], path[:, 1], 'ro-', markersize=4, label='GD Path')
52     axes[i].set_title(f'Gradient Descent for b = {b}')
53     axes[i].legend()
54
55 plt.tight_layout()
56 plt.show()

```



OBSERVATIONS

The shape gets stretched out: At first, with $b=1$, the contour lines are perfect circles. But as b gets smaller, they stretch into long, skinny ellipses. It basically turns a round bowl into a narrow valley.

The path starts zigzagging: Instead of going in a straight line to the center, the path starts "bouncing" back and forth off the walls. This happens because the y-direction is way steeper than the x-direction, so the math keeps overshooting the middle of the valley.

It slows down a lot: Even though I'm using the optimal step size from the computations at point (a), the algorithm takes way more steps to actually reach (0,0) once the function is squashed. It's a lot less efficient when the shape isn't a perfect circle.